```
#include <iostream>
#include <queue>
using namespace std;

#define V 6  // Number of landmarks

// Landmark names
string landmarks[V] = {
   "College Gate", "Library", "Canteen",
   "Playground", "Hostel", "Parking"
};

// ---------- DFS using Adjacency Matrix ----------
void DFS(int adjMatrix[V][V], int start, bool visited[]) {
   cout << landmarks[start] << " -> ";
   visited[start] = true;

   for (int i = 0; i < V; i++) {
      if (adjMatrix[start][i] == 1 && !visited[i]) {
         DFS(adjMatrix, i, visited);
      }
   }
}

// ---------- Node structure for Adjacency List ----------
struct Node {
   int data;
   Node* next;
};
```

```
// Add edge to adjacency list
void addEdge(Node* adjList[], int u, int v) {
    // Add v to u's list
    Node* newNode = new Node;
    newNode->data = v;
    newNode->next = adjList[u];
    adjList[u] = newNode;

    // Since graph is undirected, add u to v's list
    newNode = new Node;
    newNode->data = u;
    newNode->next = adjList[v];
    adjList[v] = newNode;
}

// ---------- BFS using Adjacency List ----------
void BFS(Node* adjList[], int start) {
    bool visited[V] = {false};
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        cout << landmarks[node] << " -> ";
```

```
        Node* temp = adjList[node];
        while (temp != NULL) {
            int neighbor = temp->data;
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
            temp = temp->next;
        }
    }
}

// ---------- Main Function ----------
int main() {

    // Adjacency Matrix
    int adjMatrix[V][V] = {0};

    // Adding edges
    adjMatrix[0][1] = adjMatrix[1][0] = 1;
    adjMatrix[0][2] = adjMatrix[2][0] = 1;
    adjMatrix[1][3] = adjMatrix[3][1] = 1;
    adjMatrix[2][4] = adjMatrix[4][2] = 1;
    adjMatrix[3][5] = adjMatrix[5][3] = 1;
    adjMatrix[4][5] = adjMatrix[5][4] = 1;

    // Adjacency List (array of pointers)
    Node* adjList[V];
```

```
    // Initialize list
    for (int i = 0; i < V; i++) {
        adjList[i] = NULL;
    }

    // Adding edges
    addEdge(adjList, 0, 1);
    addEdge(adjList, 0, 2);
    addEdge(adjList, 1, 3);
    addEdge(adjList, 2, 4);
    addEdge(adjList, 3, 5);
    addEdge(adjList, 4, 5);

    // DFS
    cout << "DFS Traversal (Adjacency Matrix):\n";
    bool visited[V] = {false};
    DFS(adjMatrix, 0, visited);

    cout << "\n\n";

    // BFS
    cout << "BFS Traversal (Adjacency List):\n";
    BFS(adjList, 0);

    cout << endl;

    return 0;
}
```